

C PROGRAMMING

Unit-II: Operators, Expressions & Data Input/Output

Comprehensive Study Notes

Topic 1: Operators and Expressions

1.1 What is an Operator?

An operator is a **symbol** that tells the compiler to perform specific mathematical or logical operations.

1.2 Types of Operators in C

Operator Type	Symbols
Arithmetic	+ - * / %
Unary	++ -- + - sizeof (type)
Relational	< > <= >= == !=
Logical	&& !
Assignment	= += -= *= /= %=
Conditional	?: (ternary)
Bitwise	& ^ ~ << >>

1.3 Operator Precedence

Level	Operators
1 (Highest)	() [] -> .
2	! ~ ++ -- + - * & sizeof
3	* / %
4	+ -

5	< <= > >=
6	== !=
7	&
8	^
9	
10	&&
11	
12	?:
13 (Lowest)	= += -= *= /= %=

Topic 2: Arithmetic Operators

2.1 Arithmetic Operators Table

Operator	Description	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division (Integer)	10 / 3	3
%	Modulus (Remainder)	10 % 3	1

2.2 Program: Arithmetic Operators

```
#include <stdio.h>

int main()
{
    int a = 10, b = 3;

    printf("a + b = %d\n", a + b);
    printf("a - b = %d\n", a - b);
    printf("a * b = %d\n", a * b);
    printf("a / b = %d\n", a / b);
    printf("a %% b = %d\n", a % b);

    return 0;
}
```

Output:

```
a + b = 13
a - b = 7
```

```
a * b = 30  
a / b = 3  
a % b = 1
```

2.3 Integer vs Float Division

When both operands are integers, division performs **integer division** (truncates decimal). If at least one operand is float, the result is float.

```
int a = 10, b = 3;  
float x = 10.0;  
  
printf("%d\n", a / b);    // 3 (integer division)  
printf("%.2f\n", x / b); // 3.33 (float division)
```

Topic 3: Unary Operators

3.1 Unary Operators Table

Operator	Meaning	Example
++	Increment by 1	a++ or ++a
--	Decrement by 1	a-- or --a
+	Unary plus	+a
-	Unary minus	-a
sizeof	Size of operand	sizeof(int)
(type)	Type cast	(float)a

3.2 Increment and Decrement Operators

Pre-increment (+ +a): First increment, then use the value.

Post-increment (a + +): First use the value, then increment.

3.3 Program: Increment/Decrement Operators

```
#include <stdio.h>

int main()
{
    int a = 5, b;

    // Post-increment
    b = a++;
    printf("b = a++: b = %d, a = %d\n", b, a); // b=5, a=6

    // Pre-increment
```

```
a = 5;
b = ++a;
printf("b = ++a: b = %d, a = %d\n", b, a); // b=6, a=6

// Post-decrement
a = 5;
b = a--;
printf("b = a--: b = %d, a = %d\n", b, a); // b=5, a=4

// Pre-decrement
a = 5;
b = --a;
printf("b = --a: b = %d, a = %d\n", b, a); // b=4, a=4

return 0;
}
```

Topic 4: Relational Operators

4.1 Relational Operators Table

Operator	Meaning	Example (a= 10, b=20)
<	Less than	a < b : 1 (true)
>	Greater than	a > b : 0 (false)
<=	Less than or equal	a <= b : 1 (true)
>=	Greater than or equal	a >= b : 0 (false)
==	Equal to	a == b : 0 (false)
!=	Not equal to	a != b : 1 (true)

4.2 Program: Relational Operators

```
#include <stdio.h>

int main()
{
    int a = 10, b = 20;

    printf("a < b: %d\n", a < b);    // 1
    printf("a > b: %d\n", a > b);    // 0
    printf("a == b: %d\n", a == b);  // 0
    printf("a != b: %d\n", a != b);  // 1

    return 0;
}
```

Topic 5: Logical Operators

5.1 Logical Operators Table

Operator	Meaning	Example
&&	AND - true if both true	(5>2) && (3<5) = 1
	OR - true if at least one true	(5>2) (3>5) = 1
!	NOT - reverses result	!(5>2) = 0

5.2 Program: Logical Operators

```
#include <stdio.h>

int main()
{
    int a = 1, b = 0;

    printf("a && b: %d\n", a && b); // 0
    printf("a || b: %d\n", a || b); // 1
    printf("!a: %d\n", !a);         // 0

    return 0;
}
```

Topic 6: Assignment Operators

6.1 Assignment Operators Table

Operator	Meaning	Example (a=10)
=	Simple assignment	a = 5; a becomes 5
+=	Add and assign	a += 5; a = a + 5 = 15
-=	Subtract and assign	a -= 3; a = a - 3 = 7
*=	Multiply and assign	a *= 2; a = a * 2 = 20
/=	Divide and assign	a /= 2; a = a / 2 = 5
%=	Modulus and assign	a %= 3; a = a % 3 = 1

6.2 Program: Assignment Operators

```
#include <stdio.h>

int main()
{
    int a = 10;

    printf("a = %d\n", a);    // 10
    a += 5; printf("a += 5: %d\n", a); // 15
    a -= 3; printf("a -= 3: %d\n", a); // 12
    a *= 2; printf("a *= 2: %d\n", a); // 24
    a /= 4; printf("a /= 4: %d\n", a); // 6
    a %= 4; printf("a %= 4: %d\n", a); // 2

    return 0;
}
```

Topic 7: Conditional Operator

7.1 Conditional (Ternary) Operator

Syntax: condition ? expression1 : expression2

If condition is true, expression1 is executed. If false, expression2 is executed.

7.2 Program: Conditional Operator

```
#include <stdio.h>

int main()
{
    int a = 10, b = 20, max;
    max = (a > b) ? a : b;
    printf("Maximum = %d\n", max); // 20

    int num = 7;
    printf("%s\n", (num % 2 == 0) ? "Even" : "Odd"); // Odd

    return 0;
}
```

Topic 8: Library Functions

8.1 Commonly Used Library Functions

Function	Purpose	Example
<code>sqrt(x)</code>	Square root	<code>sqrt(25) = 5</code>
<code>pow(x,y)</code>	Power	<code>pow(2,3) = 8</code>
<code>ceil(x)</code>	Round up	<code>ceil(3.14) = 4</code>
<code>floor(x)</code>	Round down	<code>floor(3.14) = 3</code>
<code>abs(x)</code>	Absolute value	<code>abs(-5) = 5</code>
<code>strlen(s)</code>	String length	<code>strlen("Hi") = 2</code>
<code>strcpy(d,s)</code>	Copy string	<code>strcpy(dest, src)</code>
<code>strcmp(s1,s2)</code>	Compare strings	<code>strcmp("a","b") = -1</code>
<code>strcat(s1,s2)</code>	Concatenate strings	<code>strcat(s1, s2)</code>
<code>toupper(c)</code>	To uppercase	<code>toupper('a') = 'A'</code>
<code>tolower(c)</code>	To lowercase	<code>tolower('A') = 'a'</code>
<code>isdigit(c)</code>	True if digit	<code>isdigit('5') = 1</code>
<code>isalpha(c)</code>	True if letter	<code>isalpha('A') = 1</code>

8.2 Program: Math Library Functions

```
#include <stdio.h>
#include <math.h>

int main()
```

```
{  
    double x = 25.0;  
  
    printf("sqrt(%.1f) = %.1f\n", x, sqrt(x));  
    printf("pow(2, 3) = %.1f\n", pow(2, 3));  
    printf("ceil(3.14) = %.1f\n", ceil(3.14));  
    printf("floor(3.14) = %.1f\n", floor(3.14));  
    printf("abs(-5) = %d\n", abs(-5));  
  
    return 0;  
}
```

Topic 9: Data Input and Output

9.1 Single Character Input - getchar()

getchar() reads a single character from the keyboard.

```
#include <stdio.h>

int main()
{
    char ch;
    printf("Enter a character: ");
    ch = getchar();
    printf("You entered: %c\n", ch);
    return 0;
}
```

9.2 Single Character Output - putchar()

putchar() displays a single character on the screen.

```
putchar('A');    // Prints A
putchar('\n');   // Prints newline
```

9.3 Formatted Input - scanf()

scanf() reads formatted input from the keyboard.

Format Specifier	Meaning
%d	Integer
%f	Float
%c	Character

%s	String (no spaces)
%lf	Double
%u	Unsigned int

9.4 Program: scanf() with Multiple Inputs

```
#include <stdio.h>

int main()
{
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    printf("Sum = %d\n", a + b);
    return 0;
}
```

9.5 Formatted Output - printf()

printf() displays formatted output on the screen.

```
int a = 10;
float b = 3.14159;

printf("Integer: %d\n", a);
printf("Float: %.2f\n", b);
printf("String: %s\n", "Hello");
```

Common Escape Sequences in printf()

Sequence	Meaning
\n	Newline
\t	Horizontal tab

\r	Carriage return
\\	Backslash
\"	Double quote

Topic 10: gets() and puts() Functions

10.1 gets() Function

gets() reads a line of text including spaces until newline is encountered.

10.2 puts() Function

puts() writes a string to the console and appends a newline.

10.3 Program: gets() and puts()

```
#include <stdio.h>

int main()
{
    char name[50];
    printf("Enter your name: ");
    gets(name);
    printf("Hello, ");
    puts(name);
    return 0;
}
```

10.4 Difference Between scanf() and gets()

scanf()	gets()
Reads until whitespace	Reads until newline
Cannot read spaces	Can read spaces
Needs & operator	No & needed (array name)
Format specifier required	No format specifier

10.5 Program: scanf() vs gets()

```
#include <stdio.h>

int main()
{
    char str1[50], str2[50];

    printf("Enter using scanf: ");
    scanf("%s", str1);
    printf("scanf read: %s\n", str1);

    // Clear input buffer
    fflush(stdin);

    printf("Enter using gets: ");
    gets(str2);
    printf("gets read: %s\n", str2);

    return 0;
}
```

Output:

```
Enter using scanf: Hello World
scanf read: Hello
Enter using gets: Hello World
gets read: Hello World
```

Topic 11: HPU Important Questions

11.1 Short Answer Questions (2-5 Marks)

1. What is an operator? Explain types of operators in C.
2. Explain arithmetic operators with example.
3. What is the difference between pre-increment and post-increment?
4. Explain relational operators with example.
5. What are logical operators? Explain with truth table.
6. What is conditional operator? Give syntax and example.
7. Explain assignment operators.
8. What is the use of sizeof operator?
9. Explain getchar() and putchar() functions.
10. Difference between scanf() and gets().

11.2 Long Answer Questions (10 Marks)

1. Explain all types of operators in C with examples.
2. Explain increment/decrement operators with examples.
3. Explain formatted input/output functions in C.
4. Write a program to demonstrate all arithmetic operations.
5. Explain commonly used library functions in C.

11.3 Program Writing Questions (5 Marks)

1. Write a program to find largest of three numbers using conditional operator.
2. Write a program to swap two numbers using assignment operators.
3. Write a program to demonstrate increment/decrement operators.
4. Write a program to find square root and power of a number.
5. Write a program to check if a character is vowel or consonant.

Objective Questions (1 Mark)

1. Which operator is used for modulus? (a) / (b) % (c) * (d) & **Answer: (b)**

2. Which is a logical operator? (a) & (b) | (c) && (d) ~ **Answer: (c)**
3. The ternary operator is: (a) ?? (b) ?: (c) :: (d) ?! **Answer: (b)**
4. ++a is called: (a) Post-increment (b) Pre-increment (c) Increment (d) Addition
Answer: (b)
5. Which function reads a character? (a) scanf (b) getchar (c) printf (d) putchar
Answer: (b)

Quick Revision

Operator Precedence

Level	Operators
1 (Highest)	() [] -> .
2	! ~ ++ -- + - * & sizeof
3	* / %
4	+ -
5	< <= > >=
6	== !=
7	&&
8	
9 (Lowest)	?: = += -= *= /=

Format Specifiers Summary

Specifier	Type
%d	int
%f	float
%c	char
%s	string
%lf	double

%u	unsigned int
%x	hexadecimal

End of Unit-II Notes